



Application Performance Monitoring & Error Tracking ด้วย Sentry

ติดตามประสิทธิภาพและตรวจจับข้อผิดพลาดของระบบงาน CRM / ERP ด้วย Sentry (Self-hosted)

หลักสูตรอบรมเชิงปฏิบัติการ (In-house Training) 3 วัน (18 ชั่วโมง) — สถาบันไอทีจีเนียส เอ็นจิเนียริ่ง

ในระบบงานสำคัญขององค์กร เช่น ระบบ CRM และ ERP การที่แอปพลิเคชันทำงานช้าหรือเกิดข้อผิดพลาดโดยไม่มีใครรู้ตัว อาจส่งผลกระทบต่อการใช้งานบริการลูกค้าและความน่าเชื่อถือขององค์กร การมีระบบ Application Performance Monitoring (APM) และ Error Tracking

ที่ดีจึงเป็นหัวใจสำคัญที่ช่วยให้ทีมพัฒนาและทีมปฏิบัติการมองเห็นปัญหาได้ก่อนที่จะใช้งานจะได้รับผลกระทบ และวินิจฉัยต้นตอของปัญหาได้อย่างรวดเร็ว

หลักสูตรนี้มุ่งเน้นการใช้งาน Sentry ซึ่งเป็นเครื่องมือ APM & Error Tracking ขั้นนำที่รองรับการติดตั้งแบบ Self-hosted ภายในองค์กรได้อย่างสมบูรณ์ จึงเหมาะอย่างยิ่งสำหรับองค์กรที่มีข้อกำหนดด้านความปลอดภัยและการเก็บข้อมูลภายใน (Data Sovereignty) เช่น สถาบันการเงิน ผู้เข้าอบรมจะได้เรียนรู้ตั้งแต่แนวคิดพื้นฐานของ Observability การติดตั้ง Sentry แบบ Self-hosted การฝังเครื่องมือเข้ากับระบบงานที่พัฒนาด้วย Spring Boot และ Angular การติดตาม Performance และ Distributed Tracing จากระดับระบบไปจนถึงฐานข้อมูล MariaDB รวมถึงการตั้งระบบแจ้งเตือน การติดตามคุณภาพของแต่ละ Release และการผนวกเข้ากับกระบวนการ CI/CD ด้วย Jenkins หรือ Kubernetes (RKE2)

เนื้อหาทั้งหมดออกแบบให้สอดคล้องกับสภาพแวดล้อมการทำงานจริงของผู้เข้าอบรม และปิดท้ายด้วย Workshop การวางระบบ Monitoring ให้กับแอปพลิเคชันจำลองแบบ CRM/ERP เพื่อให้ผู้เรียนนำไปประยุกต์ใช้กับงานจริงได้ทันที

หมายเหตุ การอบรมเป็นรูปแบบบรรยายพร้อมปฏิบัติจริง (Hands-on Workshop) โดยใช้แอปพลิเคชันจำลองที่วิทยากรเตรียมให้ผู้เรียนควรมีพื้นฐาน Java/Spring Boot และ Angular มาก่อน



ข้อมูลหลักสูตร

วัตถุประสงค์ของหลักสูตร

- เข้าใจแนวคิดของ Observability และ Application Performance Monitoring (APM) อย่างถูกต้อง
- เข้าใจความเชื่อมโยงของ Errors, Performance (Traces) และ Logs/Context
- ติดตั้งและตั้งค่า Sentry แบบ Self-hosted ภายในองค์กรได้ด้วยตนเอง
- ฝังเครื่องมือ Sentry เข้ากับแอปพลิเคชัน Spring Boot และ Angular ได้
- ตรวจสอบ จัดกลุ่ม และวินิจฉัยข้อผิดพลาด (Error / Issue Tracking) ได้อย่างเป็นระบบ
- ติดตามประสิทธิภาพด้วย Performance Monitoring และ Distributed Tracing ข้ามชั้น Frontend, Backend และ Database
- ระบุ Query ของ MariaDB ที่ทำงานช้า และคงวด้านประสิทธิภาพได้
- ตั้งค่าระบบแจ้งเตือน (Alerting) และช่องทางการแจ้งเตือนให้เหมาะสมกับทีม
- ติดตามคุณภาพของแต่ละ Release (Release Health) และใช้งาน Session Replay เพื่อสืบสวนปัญหา
- ผนวกการ Monitoring เข้ากับกระบวนการ CI/CD ด้วย Jenkins และการ Deploy บน Kubernetes (RKE2)
- วางระบบ Monitoring ให้กับระบบงานแบบ CRM/ERP ได้ครบวงจรตั้งแต่ต้นจนจบ

กลุ่มเป้าหมาย

- นักพัฒนาซอฟต์แวร์ (Software Developer) ที่ดูแลระบบงาน CRM / ERP
- ทีม DevOps และผู้ดูแลระบบ (System / Infrastructure Engineer)
- Technical Lead และ Solution Architect ที่ต้องการวางมาตรฐานการ Monitoring
- ผู้ดูแลคุณภาพและความเสถียรของระบบงานสำคัญขององค์กร

ความรู้พื้นฐานที่ผู้เข้าอบรมควรมี

- มีพื้นฐานการเขียนโปรแกรมภาษา Java และการพัฒนาด้วย Spring Boot
- มีพื้นฐานการพัฒนา Frontend ด้วย Angular / TypeScript / JavaScript
- เข้าใจการทำงานของ HTTP, REST API และฐานข้อมูลเชิงสัมพันธ์ (MariaDB / SQL)
- สามารถใช้งาน Command Line บน Linux และ Docker เบื้องต้นได้
- เข้าใจแนวคิดการ Deploy แอปพลิเคชันบน Container / Kubernetes เบื้องต้น (จะดีมาก)



เครื่องมือและเทคโนโลยีที่ใช้ในหลักสูตร

- **Sentry (Self-hosted):** เครื่องมือ APM & Error Tracking หลักของหลักสูตร
- **Java + Spring Boot:** ระบบ Backend (ใช้ sentry-spring-boot-starter, sentry-jdbc)
- **Angular + TypeScript:** ระบบ Frontend (ใช้ @sentry/angular)
- **MariaDB:** ฐานข้อมูลของระบบงาน
- **Docker (v28.x) และ Docker Compose:** สำหรับติดตั้ง Self-hosted Sentry
- **Kubernetes (RKE2 v1.33):** สำหรับ Deploy ระบบงานและ Sentry ไปยัง Production
- **Jenkins:** ระบบ CI/CD สำหรับพัฒนาก่อน Release และ Source Maps
- **Linux (CentOS 8):** ระบบปฏิบัติการของ Server
- **แอประบบงานจำลอง CRM/ERP (Spring Boot + Angular):** สำหรับใช้ทำ Workshop ตลอดคอร์ส

ภาพรวมหลักสูตร 3 วัน

วันที่	หัวข้อหลัก
วันที่ 1	ปูพื้นฐาน Observability & APM, ติดตั้ง Self-hosted Sentry และเริ่ม Instrument Spring Boot
วันที่ 2	Error Tracking เชิงลึก, Performance Monitoring และ Distributed Tracing (Spring Boot → MariaDB → Angular)
วันที่ 3	Alerting, Release Health, Session Replay, แผน Jenkins CI/CD + Kubernetes และ Workshop ปิดท้าย



รายละเอียดเนื้อหาการอบรม

วันที่ 1: พื้นฐาน Observability & APM และเริ่มต้นใช้งาน Sentry

เข้าใจแนวคิดการ Monitoring ระบบงาน ติดตั้ง Sentry แบบ Self-hosted ภายในองค์กร และฝังเครื่องมือเข้ากับ Spring Boot ตัวแรก

Module 1.1 ทำความรู้จัก Observability และ APM

- ความท้าทายของการดูแลระบบงานสำคัญอย่าง CRM / ERP ในยุคปัจจุบัน
- Monitoring กับ Observability ต่างกันอย่างไร
- **เสาหลักของ Observability:** Metrics, Logs และ Traces
- บทบาทของ APM (Application Performance Monitoring) และ Error Tracking
- ภาพรวมเครื่องมือในตลาด (Self-hosted vs SaaS) และเหตุผลที่เลือก Sentry

Module 1.2 รู้จัก Sentry และสถาปัตยกรรม

- Sentry คืออะไร และครอบคลุมงานด้านใดบ้าง (Errors, Performance, Replay, Release)
- **สถาปัตยกรรมของ Sentry:** SDK → DSN → Ingest → UI
- **แนวคิดสำคัญ:** Organization, Project, Environment, Release และ DSN
- ขอบเขตที่ Sentry ทำได้ดี และส่วนที่ควรเสริมด้วยเครื่องมืออื่น (Infra/K8s Metrics)
- ภาพรวม Workflow การใช้งานจริงในทีมพัฒนา

Module 1.3 ติดตั้ง Self-hosted Sentry ภายในองค์กร

- ข้อกำหนดของระบบ (System Requirements) และข้อควรระวังบน CentOS 8
- การติดตั้ง Self-hosted Sentry ด้วย Docker Compose
- ภาพรวมส่วนประกอบของระบบ (Web, Worker, Relay, PostgreSQL, Redis, Kafka)
- **การตั้งค่าเริ่มต้น:** สร้างผู้ดูแลระบบ และการเข้าถึงผ่าน Reverse Proxy
- การบริหารจัดการผู้ใช้ ทีม และสิทธิ์การเข้าถึง (Roles & Permissions)



Module 1.4 ตั้งค่าโปรเจกต์และแนวคิดหลักของ Sentry

- การสร้าง Project สำหรับระบบ CRM / ERP
- ทำความเข้าใจ DSN และการเชื่อมต่อกับแอปพลิเคชัน
- **การแยกสภาพแวดล้อม (Environment):** development / staging / production
- แนวคิด Release และความสำคัญต่อการติดตามปัญหา
- การกำหนด Data Scrubbing เพื่อปกป้องข้อมูลส่วนบุคคล (PII) — สำคัญมากสำหรับข้อมูลธนาคาร

Module 1.5 เริ่ม Instrument ระบบ Backend ด้วย Spring Boot

- การเพิ่ม Dependency sentry-spring-boot-starter
- การตั้งค่า DSN, Environment และ Release ผ่าน application.properties / application.yml
- การทดสอบส่ง Error ตัวแรกเข้า Sentry
- การจับ Unhandled Exception โดยอัตโนมัติ
- **Workshop:** เชื่อมต่อแอป Spring Boot ของระบบงานจำลองเข้ากับ Sentry และยืนยันว่าข้อมูลเข้าระบบ

วันที่ 2: Error Tracking เชิงลึก, Performance Monitoring และ Distributed Tracing

วิทยากรจะนำผู้เรียนไปปฏิบัติจริงเป็นระบบ ติดตามประสิทธิภาพข้ามชั้นระบบ และใช้ Trace ตั้งแต่หน้าจอก่อน Angular จนถึง Query บน MariaDB

Module 2.1 Error & Issue Tracking เชิงลึก

- กลไกการรวบรวมและจัดกลุ่มข้อผิดพลาด (Issue Grouping & Fingerprinting)
- การอ่าน Stack Trace และข้อมูลประกอบของแต่ละ Issue
- **สถานะของ Issue:** Unresolved, Resolved, Ignored และ Regression
- การกำหนดความรุนแรง (Level) และการมอบหมายผู้รับผิดชอบ (Ownership Rules)
- การค้นหาและกรอง Issue ด้วย Search Query

Module 2.2 การเพิ่มบริบทให้กับข้อผิดพลาด (Context Enrichment)

- การแนบข้อมูลผู้ใช้ (User Context) เพื่อระบุว่าใครได้รับผลกระทบ
- การใช้ Tags สำหรับจัดหมวดหมู่และกรองข้อมูล
- การใช้ Breadcrumbs เพื่อย้อนรอยเหตุการณ์ก่อนเกิด Error
- การแนบ Custom Context และ Extra Data



- การจับ Error ด้วยตนเอง (captureException, captureMessage)

Module 2.3 แนวคิด Performance Monitoring และ Distributed Tracing

- Transaction และ Span คืออะไร
- **การวัดประสิทธิภาพ:** Throughput, Latency (P50/P75/P95) และ Failure Rate
- แนวคิด Distributed Tracing และการส่งต่อ Trace ข้ามบริการ (Trace Propagation)
- การตั้งค่า traces-sample-rate และกลยุทธ์การ Sampling ในระบบ Production
- การอ่านหน้า Performance และ Trace View ใน Sentry

Module 2.4 Performance Tracing ใน Spring Boot และ MariaDB

- การเปิดใช้งาน Performance Monitoring ใน Spring Boot
- การสร้าง Transaction อัตโนมัติสำหรับ HTTP Request
- การใช้ @SentryTransaction และ @SentrySpan เพื่อระบุช่วงงานที่สนใจ
- การติดตาม Query ฐานข้อมูลด้วยโมดูล sentry-jdbc (เปลี่ยนทุก Query เป็น Span)
- **Workshop:** ค้นหา Query MariaDB ที่ทำงานช้าและปัญหา N+1 จากระบบงานจำลอง

Module 2.5 Instrument ระบบ Frontend ด้วย Angular

- การติดตั้งและตั้งค่า @sentry/angular
- การจับ Error ฝั่ง Frontend และการตั้งค่า ErrorHandler ของ Angular
- การติดตามประสิทธิภาพของ Component (Component Tracking)
- การติดตาม Routing และการโหลดหน้า (Page Load / Navigation)
- การตั้งค่า Environment และ Release ให้สอดคล้องกับฝั่ง Backend

Module 2.6 เชื่อม Distributed Tracing ระหว่าง Frontend และ Backend

- การส่งต่อ Trace จาก Angular → Spring Boot ด้วย Trace Header (sentry-trace, baggage)
- การตั้งค่า tracePropagationTargets ให้ครอบคลุม API ของระบบงาน
- **การมองเห็น Trace แบบ End-to-End:** ผู้ใช้คลิก → API → Query ฐานข้อมูล
- การวินิจฉัยว่าความช้าเกิดขึ้นที่ชั้นใดของระบบ
- **Workshop:** ไล่ Trace หนึ่งรายการตั้งแต่หน้าจอ CRM/ERP จำลองจนถึง MariaDB



วันที่ 3: Alerting, Release Health, การผนวก CI/CD และ Workshop ปิดท้าย

ตั้งระบบแจ้งเตือนที่ใช้งานได้จริง ติดตามคุณภาพแต่ละ Release ผนวกเข้ากับ Jenkins และ Kubernetes แล้วลงมือวางระบบ Monitoring ให้ระบบงานจำลองแบบครบวงจร

Module 3.1 ระบบแจ้งเตือน (Alerting & Notifications)

- **ประเภทของ Alert:** Issue Alerts และ Metric Alerts
- การตั้งเงื่อนไขการแจ้งเตือน (Conditions, Thresholds, Frequency)
- ช่องทางการแจ้งเตือน (Email, Slack, Microsoft Teams, Webhook)
- การออกแบบ Alert ให้มีประโยชน์และลด Alert Fatigue
- แนวคิดการเชื่อมโยง Alert กับ SLA / SLO ของระบบงาน

Module 3.2 Dashboards และการวิเคราะห์ข้อมูล (Discover)

- การสร้าง Dashboard เพื่อภาพรวมสุขภาพของระบบ CRM / ERP
- การใช้งาน Discover เพื่อสืบค้นและวิเคราะห์ข้อมูลเชิงลึก
- **Widget สำคัญ:** Error Rate, Apdex, Slow Transactions, Top Issues
- การแชร์ Dashboard ให้ทีมและผู้บริหาร
- การตั้งค่านับมองสำหรับแต่ละบทบาท (Developer / Ops / Management)

Module 3.3 Release Health และ Source Maps

- **แนวคิด Release Health:** Crash Free Rate, Adoption และ Regression
- การสร้างและติดตาม Release ให้กับแต่ละรอบการ Deploy
- การเชื่อมโยง Commit เพื่อระบุว่าโค้ดส่วนใดทำให้เกิดปัญหา (Suspect Commits)
- การอัปโหลด Source Maps ของ Angular เพื่อให้เห็น Stack Trace ที่อ่านได้
- การติดตามว่า Release ใหม่ดีขึ้นหรือแย่ลงเทียบกับรอบก่อน

Module 3.4 Session Replay และ Profiling

- การใช้งาน Session Replay เพื่อย้อนดูสิ่งที่ผู้ใช้ทำก่อนเกิดปัญหา
- การตั้งค่า Privacy / Masking เพื่อปกป้องข้อมูลสำคัญใน Replay
- แนวคิด Profiling สำหรับเจาะลึกประสิทธิภาพระดับโค้ด



- การเชื่อมโยง Replay เข้ากับ Issue และ Trace
- ข้อควรพิจารณาด้านความเป็นส่วนตัวสำหรับระบบงานธนาคาร

Module 3.5 ผนวก Monitoring เข้ากับ CI/CD (Jenkins) และ Kubernetes

- การติดตั้ง sentry-cli และการใช้งานใน Pipeline
- การสร้าง Release และอัปโหลด Source Maps อัตโนมัติใน Jenkins Pipeline
- การส่งข้อมูล Deploy (Associate Commits & Deploys) จาก Jenkins ไปยัง Sentry
- แนวทาง Deploy Self-hosted Sentry หรือเชื่อมต่อบน Kubernetes (RKE2)
- การจัดการ DSN และ Secrets อย่างปลอดภัยใน Pipeline และ Kubernetes
- **Workshop:** เพิ่มขั้นตอนสร้าง Release และอัปโหลด Source Maps ลงใน Jenkins Pipeline จำลอง

Module 3.6 Workshop / โครงการปิดท้าย (Capstone Project)

- **โจทย์:** วางระบบ Monitoring ครบวงจรให้กับแอปพลิเคชันจำลองแบบ CRM/ERP (Spring Boot + Angular + MariaDB)
- ฝัง Sentry ทั้งฝั่ง Backend และ Frontend พร้อมตั้งค่า Environment และ Release
- เปิดใช้งาน Performance Monitoring และ Distributed Tracing แบบ End-to-End
- จำลองสถานการณ์ Incident (Error และ Slow Query) แล้ววินิจฉัยหาต้นตอ
- ตั้งค่า Alert, Dashboard และ Release Health ให้พร้อมใช้งานจริง
- นำเสนอผลการวินิจฉัยและแนวทางแก้ไขปัญหา
- **ต่อยอด:** แนวทางเสริม Infrastructure & Kubernetes Monitoring ด้วย Prometheus + Grafana

หมายเหตุและเงื่อนไข

- เนื้อหาและลำดับการสอนสามารถปรับเปลี่ยนได้ตามความเหมาะสมและพื้นฐานของผู้เข้าอบรม
- การอบรมเป็นรูปแบบบรรยายพร้อมปฏิบัติจริง (Hands-on Workshop) โดยใช้แอปพลิเคชันจำลองที่วิทยากรเตรียมให้
- ผู้เข้าอบรมควรเตรียม Notebook ระบบ Windows หรือ macOS ที่ติดตั้ง Docker Desktop, JDK, Node.js และ IDE มาด้วย
- ควรเตรียมสภาพแวดล้อมสำหรับ Lab เช่น สิทธิ์เข้าถึง Cluster Kubernetes (RKE2) หรือใช้ Docker บนเครื่องผู้เรียนทดแทน
- Self-hosted Sentry แนะนำระบบฐาน Debian/Ubuntu จึงควรทดสอบการติดตั้งบน CentOS 8 ล่วงหน้า หรือใช้รูปแบบ Container บน Kubernetes แทน